

Day 82: Database Fundamentals

1. Day Number

Day 82

2. Topic Name

Databases, Tables, Rows, Columns, and Keys

Today you will learn how applications store structured information in a **database** instead of keeping everything only in files or DataFrames.

3. Connection

Until now, you have mainly worked with data stored in:

- CSV files
- Excel files
- JSON files
- Pandas DataFrames

These are useful for learning, analysis, and exchanging data.

But applications often need data to remain stored permanently and be easy to search, update, and connect.

That is where **databases** become useful.

A simple progression is:

```
Files
  ↓
DataFrames
  ↓
Databases
```

4. Important Topics

Database

A **database** is an organized collection of data.

For example, an online shopping application might store:

```
Customers
Products
Orders
Payments
```

inside a database.

Table

A **table** stores one type of structured information.

For example:

customers

could store information about customers.

A table looks similar to a spreadsheet or Pandas DataFrame.

Row

A **row** represents one record.

For example:

1 Asha Bengaluru

could represent one customer.

Column

A **column** represents one type of information.

For a customer table, columns might be:

customer_id
name
city

Primary key

A **primary key** uniquely identifies each row.

For example:

customer_id

could be the primary key.

That means two customers should not have the same `customer_id`.

Relational database

A **relational database** stores data in tables that can be related to one another.

For example:

```
customers
orders
```

An order may belong to a particular customer.

You do not need to learn complex table relationships yet. For today, understand that relational databases organize structured information into tables.

5. Foundational Notes

Think of a database table like a well-organized spreadsheet.

Example:

customer_id	name	city
1	Asha	Bengaluru
2	Ravi	Mysuru
3	Neha	Chennai

Here:

```
Table:
customers

Columns:
customer_id
name
city

Rows:
3 customers
```

Each row represents one customer.

Each column describes one property of the customer.

The `customer_id` gives each customer a unique identity.

6. File Storage vs Database Storage

Suppose an application stores customers in a CSV file:

```
customers.csv
```

That can work for a very small task.

But imagine an application eventually contains:

```
100,000 customers
many orders
frequent updates
multiple users
```

Managing everything through individual files becomes harder.

File storage

Files such as CSV and JSON are useful when you want to:

- save simple data
- share data
- perform analysis
- work with small datasets
- move data between programs

Example:

```
customers.csv
```

Database storage

A database is useful when an application needs to:

- store data persistently
- search records
- add new records
- update existing records
- delete records
- connect related information

For example:

```
Application
  ↓
Database
  ↓
customers table
orders table
```

A database does not replace files in every situation. They solve different problems.

7. Easy Example: Customers and Orders

Imagine a small shopping application.

It needs to remember its customers.

The `customers` table might contain:

customer_id	name	city
1	Meera	Bengaluru
2	Arjun	Pune
3	Sara	Hyderabad

The application also has orders.

A simple conceptual `orders` table might look like:

order_id	customer_id	product
101	1	Keyboard
102	3	Mouse
103	1	Monitor

Notice this value:

```
customer_id = 1
```

appears in the orders table.

It tells us that the order belongs to customer 1.

For example:

```
Customer 1  
Meera  
  ↓  
Order 101  
Order 103
```

This is the basic idea behind a **relational database**: information can be stored in separate tables and connected through meaningful values.

For Day 82, you only need to understand the idea.

8. Problem Statement

Design a very small database table called:

```
customers
```

It should contain exactly these main pieces of information:

```
customer ID
customer name
customer city
```

Your goal is to decide:

- what the table represents
- what its columns should be
- what one row represents
- which column should uniquely identify a customer

Do **not** create a complicated database.

9. Concepts Used

You will practice:

```
database
table
row
column
record
primary key
unique identifier
relational database
structured data
```

You are mainly designing the structure today rather than writing advanced database commands.

10. Thought Process

Start with this question:

What type of thing am I storing?

Customers.

Therefore, a good table name is:

```
customers
```

Next ask:

What information do I need about each customer?

For this simple exercise:

```
customer ID
name
city
```

So those become columns.

Then ask:

What does one row represent?

One customer.

For example:

```
1 | Priya | Bengaluru
```

means:

```
customer_id → 1
name        → Priya
city        → Bengaluru
```

Finally ask:

How can I uniquely identify each customer?

Use:

```
customer_id
```

as the primary key.

11. Beginner-Friendly Table Design

A simple design could be:

```
Table name:
customers
```

Column	Meaning
customer_id	Unique number for the customer
name	Customer's name
city	Customer's city

Conceptually:

```

customers
-----
customer_id | name   | city
-----
1           | Priya  | Bengaluru
2           | Rahul  | Mumbai
3           | Kavya  | Chennai

```

The important rule is:

```
customer_id must identify one unique customer
```

So:

```

1 → Priya
2 → Rahul
3 → Kavya

```

is fine.

But this would be problematic:

```

1 → Priya
1 → Rahul

```

because two rows are using the same primary key.

12. Suggested Solving Approach

Use a **simple relational database concept**.

Think in this order:

```

What am I storing?
    ↓
Customers

What table represents it?
    ↓
customers

What information does each customer need?
    ↓
customer_id
name
city

What does one row represent?

```

↓
One customer

Which value uniquely identifies that row?
↓
customer_id

For today, the main goal is to become comfortable thinking about data as **tables containing structured records**.

13. Easy Edge Cases

Duplicate ID

Suppose you have:

customer_id	name
1	Aman
1	Riya

This creates a problem because:

customer_id = 1

no longer identifies exactly one customer.

A primary key should be **unique**.

Empty value

Suppose a customer row contains:

4	Kiran	
---	-------	--

The city value is empty.

Whether this should be allowed depends on the application's requirements.

For example, an application might decide:

name	→ required
city	→ optional

The important idea is that database design should reflect what information your application actually requires.

14. Expected Table

Your final conceptual table should look similar to:

```

customers
-----
customer_id | name      | city
-----
1           | Asha     | Bengaluru
2           | Vikram   | Delhi
3           | Neha     | Chennai

```

You should be able to explain:

Database:
stores structured information

Table:
customers

One row:
one customer

Columns:
customer_id, name, city

Primary key:
customer_id

At this stage, you do **not** need multiple related tables, advanced SQL, normalization, indexing, or database architecture.

15. Hint Only

Start by drawing the table on paper or writing it as plain text:

```

customers
-----
customer_id | name | city
-----

```

Then add three example customers.

After that, ask yourself:

Can two customers have the same customer_id?

Your answer should be **no**, because `customer_id` is meant to uniquely identify each customer.

The main idea to remember from Day 82 is:

Database



Table



Rows and Columns



Primary Key identifies each row

Tomorrow, when you start working with database commands, this table structure will make much more sense.